Asynchronous Programming in JavaScript

As a JavaScript developer, understanding **asynchronous programming** is critical because JavaScript runs on a **single-threaded event loop model**. That means it can execute only one task at a time — yet it handles timers, API calls, file reads, user interactions, etc., without blocking the UI.

What is Asynchronous Programming?

Asynchronous programming allows JavaScript to execute long-running tasks (like API calls, file reading, timers) **without blocking the main thread**.

JavaScript is:

✅ Single-threaded

✅ Synchronous by default

✅ Uses Event Loop for async behavior

Why Do We Need Async Programming?

Imagine this:

```javascript
console.log("Start");

for (let i = 0; i < 10000000000; i++) {
   // Heavy task
}

console.log("End");
```

👉 Browser freezes.

Now imagine calling an API that takes 3 seconds.

Without async, everything waits.

JavaScript

```javascript
console.log("Start");

setTimeout(() => {
  console.log("Inside Timeout");
}, 2000);

console.log("End");
```

Output:

Code

```
Start
End
Inside Timeout
```

↓

👉 Even though `setTimeout` appears in between, it does not block execution.

With async:

- UI stays responsive
- Other code continues executing

Types of Asynchronous Techniques in JavaScript

There are 3 major approaches:

1. **Callbacks**
2. **Promises**
3. **Async/Await (Modern & Recommended)**

Method 1: Callbacks

What is a Callback?

A function passed as an argument to another function.

Practical Demo 1 – setTimeout (Basic Async)

```javascript
console.log("Start");

setTimeout(function () {
    console.log("Inside Timeout");
}, 2000);

console.log("End");
```

Output:

```
Start
End
Inside Timeout
```

Why?

- setTimeout is handled by browser Web API
- It does not block the main thread

Real-Time Demo 2 – Simulating API Call

```javascript
function fetchData(callback) {
    console.log("Fetching data...");

    setTimeout(function () {
        callback("Data received");
    }, 2000);
}

fetchData(function (data) {
    console.log(data);
});
```

Problem: Callback Hell 😱

JavaScript

```javascript
login(function () {
    getUser(function () {
        getOrders(function () {
            processPayment(function () {
                console.log("Done");
            });
        });
    });
});
```

👉 Hard to read
👉 Hard to debug
👉 Hard to maintain

Method 2: Promises

A **Promise** represents a value that may be available:

- Pending
- Resolved (fulfilled)
- Rejected

Creating a Promise

JavaScript

```javascript
let promise = new Promise((resolve, reject) => {
    let success = true;

    setTimeout(() => {
        if(success)
            resolve("Data Loaded");
        else
            reject("Error occurred");
    }, 2000);
});
```

Consuming a Promise

```javascript
promise
    .then(result => {
        console.log(result);
    })
    .catch(error => {
        console.log(error);
    });
```

Method 3: Async / Await (Modern Approach)

Introduced in ES2017.

Why Async/Await?

- Cleaner syntax
- Looks like synchronous code
- Easier to read
- Easier to debug

Basic Syntax

```javascript
async function fetchData() {
    return "Hello";
}
```

👉 Async function always returns a Promise.

Using await

```javascript
function getData() {
    return new Promise(resolve => {
        setTimeout(() => resolve("Data received"), 2000);
    });
}

async function displayData() {
    let data = await getData();
    console.log(data);
}

displayData();
```

When to Use What?

Scenario	Recommended Approach
Simple async	Callback
Complex logic	Promise
Production code	Async/Await